Universidad CENFOTEC

Escuela de Ingeniería del Software

Proyecto Misión Órbita
Segundo avance: Ejecución de la misión

Curso:	SOFT-01 Principios de programación 1
Sección:	SCV6
Periodo:	C3-2026
Docente facilitador:	Verónica Mora Lezcano
Estudiante:	Tayrin Alexa Ramírez Barrantes (trabajo individual)
Ejercicios entregados:	7 y 8
Fecha de entrega:	8 de noviembre de 2026, 23:59
Repositorio Git:	https://github.com/Tay20RB-ux/mision-orbita
Rama a revisar:	main

Correcciones del Avance 1

Tabla de cambios con la retroalimentación del docente.

Observación del docente	Corrección aplicada	Commit
Sin observaciones: el Avance 1 obtuvo la calificación máxima.	No hubo errores que corregir. En este avance se mantienen las mismas fórmulas, el encabezado de cada archivo, los nombres en snake_case, las constantes en MAYÚSCULAS y los comentarios del Avance 1.	No aplica

Ejercicio 7: Actividades de exploración científica

Archivo: Ejercicio7Exploracion.py
Tipo de estructura: Ciclo while con variable de control, condicional múltiple y condicional anidada.

1. Tabla de variables

Nombre	Tipo de dato	Descripción	Clasificación	Valor inicial / restricción
OPCION_ANALIZAR_TERRENO	int	Número de la opción Analizar terreno.	Constante	1
OPCION_RECOLECTAR_MUESTRA	int	Número de la opción Recolectar muestra.	Constante	2
OPCION_REALIZAR_OBSERVACION	int	Número de la opción Realizar observación.	Constante	3
OPCION_FINALIZAR	int	Número de la opción Finalizar exploración.	Constante	4
ENERGIA_ANALIZAR_TERRENO	int	Energía que gasta Analizar terreno.	Constante	8
PROVISIONES_ANALIZAR_TERRENO	int	Provisiones que gasta Analizar terreno.	Constante	0
PUNTOS_ANALIZAR_TERRENO	int	Puntos que da Analizar terreno.	Constante	10
ENERGIA_RECOLECTAR_MUESTRA	int	Energía que gasta Recolectar muestra.	Constante	12
PROVISIONES_RECOLECTAR_MUESTRA	int	Provisiones que gasta Recolectar muestra.	Constante	2
PUNTOS_RECOLECTAR_MUESTRA	int	Puntos que da Recolectar muestra.	Constante	20
ENERGIA_REALIZAR_OBSERVACION	int	Energía que gasta Realizar observación.	Constante	5
PROVISIONES_REALIZAR_OBSERVACION	int	Provisiones que gasta Realizar observación.	Constante	0
PUNTOS_REALIZAR_OBSERVACION	int	Puntos que da Realizar observación.	Constante	8
energia_disponible	int	Energía de la nave; se actualiza con cada actividad.	Entrada	Dato del usuario; entero positivo
provisiones_disponibles	int	Provisiones de la nave; se actualizan con cada actividad.	Entrada	Dato del usuario; entero positivo
opcion	int	Opción del menú que elige el usuario en cada repetición.	Entrada	Dato del usuario; válidas de 1 a 4
puntos_cientificos	int	Total de puntos de las actividades realizadas.	Salida (acumulador)	0
actividades_realizadas	int	Cantidad de actividades realizadas.	Salida (contador)	0
muestras_recolectadas	int	Cantidad de muestras recolectadas.	Salida (contador)	0
explorando	bool	Indica si el menú sigue repitiéndose.	Intermedia (variable de control)	True; pasa a False al terminar
nombre_actividad	str	Nombre de la actividad elegida.	Intermedia	Se asigna según la opción
energia_actividad	int	Energía que gasta la actividad elegida.	Intermedia	Se asigna según la opción
provisiones_actividad	int	Provisiones que gasta la actividad elegida.	Intermedia	Se asigna según la opción
puntos_actividad	int	Puntos que da la actividad elegida.	Intermedia	Se asigna según la opción

2. Diagrama de flujo - Ejercicio 7

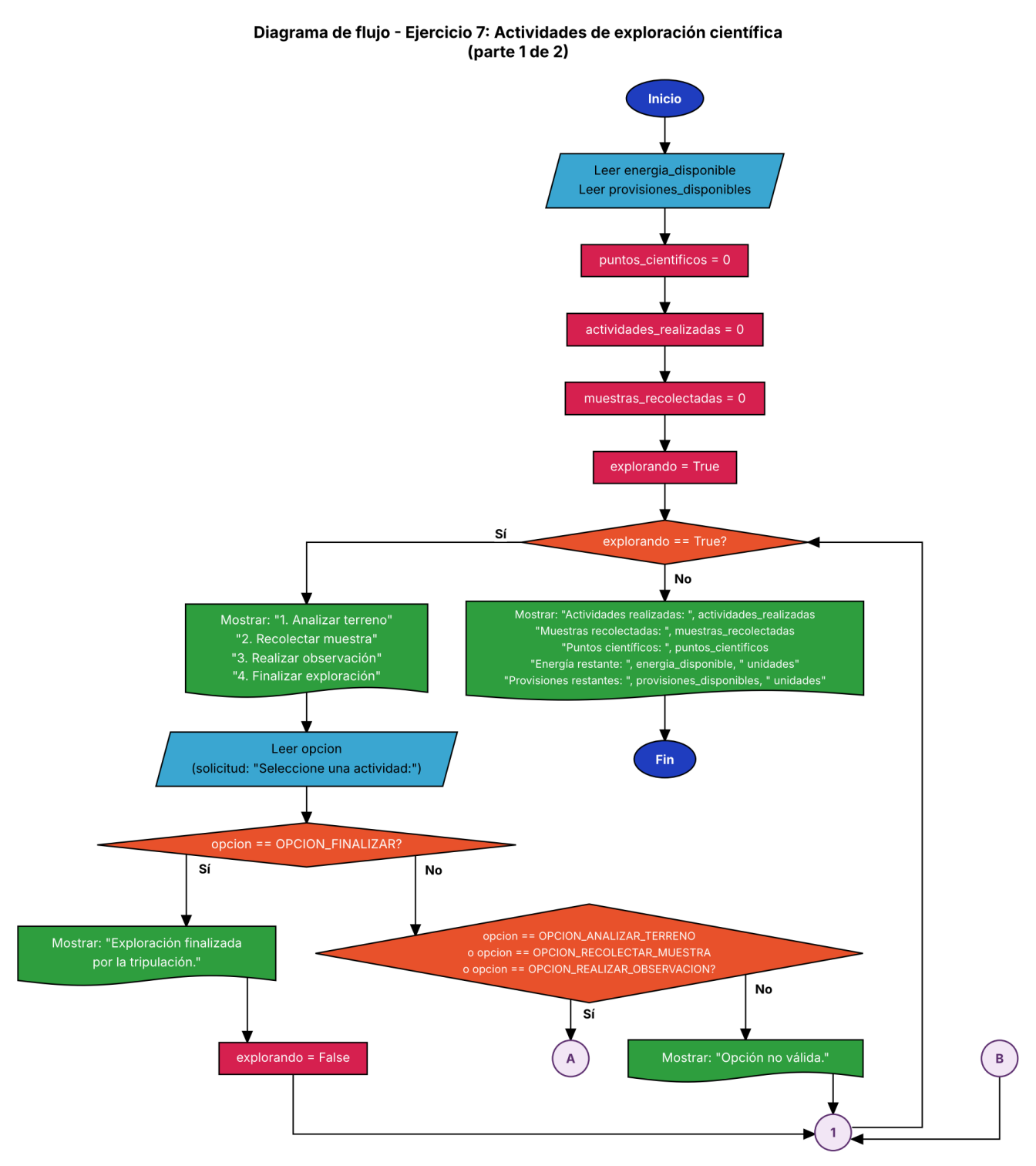


Diagrama de flujo - Ejercicio 7: Actividades de exploración científica (parte 2 de 2)



3. Código

Capturas del código en Visual Studio Code.

PY Ejercicio7Exploracion.py

```python
# ============================================================
# Archivo: Ejercicio7Exploracion.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV6
# Integrantes: Tayrin Alexa Ramírez Barrantes
# Fecha: 05/11/2026       Versión: 1.0
# Descripción: Menú repetido de actividades de exploración científica.
# ============================================================

# --- Declaración de constantes ---
# Opciones del menú y costos de cada actividad (unidades ficticias de misión).
OPCION_ANALIZAR_TERRENO = 1
OPCION_RECOLECTAR_MUESTRA = 2
OPCION_REALIZAR_OBSERVACION = 3
OPCION_FINALIZAR = 4

ENERGIA_ANALIZAR_TERRENO = 8
PROVISIONES_ANALIZAR_TERRENO = 0
PUNTOS_ANALIZAR_TERRENO = 10

ENERGIA_RECOLECTAR_MUESTRA = 12
PROVISIONES_RECOLECTAR_MUESTRA = 2
PUNTOS_RECOLECTAR_MUESTRA = 20

ENERGIA_REALIZAR_OBSERVACION = 5
PROVISIONES_REALIZAR_OBSERVACION = 0
PUNTOS_REALIZAR_OBSERVACION = 8

# --- Entrada de datos ---
energia_disponible = int(input("Ingrese la energía disponible al llegar al destino: "))
provisiones_disponibles = int(input("Ingrese las provisiones disponibles al llegar al destino: "))

# --- Inicialización del estado de la exploración ---
```

PY Ejercicio7Exploracion.py

```python
# Contadores y acumulador empiezan en 0; la variable de control permite el ciclo.
puntos_cientificos = 0
actividades_realizadas = 0
muestras_recolectadas = 0
explorando = True

# --- Ciclo del menú ---
# El ciclo termina solo cuando explorando pasa a False (opción 4 o recursos insuficientes).
while explorando:
    print("1. Analizar terreno")
    print("2. Recolectar muestra")
    print("3. Realizar observación")
    print("4. Finalizar exploración")
    opcion = int(input("Seleccione una actividad: "))

    # --- Decisión múltiple sobre la opción elegida ---
    if opcion == OPCION_FINALIZAR:
        print("Exploración finalizada por la tripulación.")
        explorando = False
    elif (opcion == OPCION_ANALIZAR_TERRENO
          or opcion == OPCION_RECOLECTAR_MUESTRA
          or opcion == OPCION_REALIZAR_OBSERVACION):
        # Primero se obtienen los datos de la actividad elegida.
        if opcion == OPCION_ANALIZAR_TERRENO:
            nombre_actividad = "Analizar terreno"
            energia_actividad = ENERGIA_ANALIZAR_TERRENO
            provisiones_actividad = PROVISIONES_ANALIZAR_TERRENO
            puntos_actividad = PUNTOS_ANALIZAR_TERRENO
        elif opcion == OPCION_RECOLECTAR_MUESTRA:
            nombre_actividad = "Recolectar muestra"
            energia_actividad = ENERGIA_RECOLECTAR_MUESTRA
            provisiones_actividad = PROVISIONES_RECOLECTAR_MUESTRA
```

PY Ejercicio7Exploracion.py

```python
            puntos_actividad = PUNTOS_RECOLECTAR_MUESTRA
        else:
            nombre_actividad = "Realizar observación"
            energia_actividad = ENERGIA_REALIZAR_OBSERVACION
            provisiones_actividad = PROVISIONES_REALIZAR_OBSERVACION
            puntos_actividad = PUNTOS_REALIZAR_OBSERVACION

        # Se compara con >= porque tener justo lo necesario sí alcanza.
        if energia_disponible >= energia_actividad and provisiones_disponibles >= provisiones_actividad:
            energia_disponible = energia_disponible - energia_actividad
            provisiones_disponibles = provisiones_disponibles - provisiones_actividad
            actividades_realizadas = actividades_realizadas + 1
            puntos_cientificos = puntos_cientificos + puntos_actividad
            # Solo la recolección de muestras aumenta este contador.
            if opcion == OPCION_RECOLECTAR_MUESTRA:
                muestras_recolectadas = muestras_recolectadas + 1
            print("Actividad realizada: " + nombre_actividad + ". Puntos obtenidos: " + str(puntos_actividad))
        else:
            # El estado no cambia y el menú termina aunque otra actividad más barata sí alcanzara.
            print("Recursos insuficientes para realizar la actividad. La exploración finaliza.")
            explorando = False
    else:
        # Opción inválida: no cambia nada y el menú se muestra de nuevo.
        print("Opción no válida.")

# --- Salida del resumen final ---
print("Actividades realizadas: " + str(actividades_realizadas))
print("Muestras recolectadas: " + str(muestras_recolectadas))
print("Puntos científicos: " + str(puntos_cientificos))
print("Energía restante: " + str(energia_disponible) + " unidades")
print("Provisiones restantes: " + str(provisiones_disponibles) + " unidades")
```

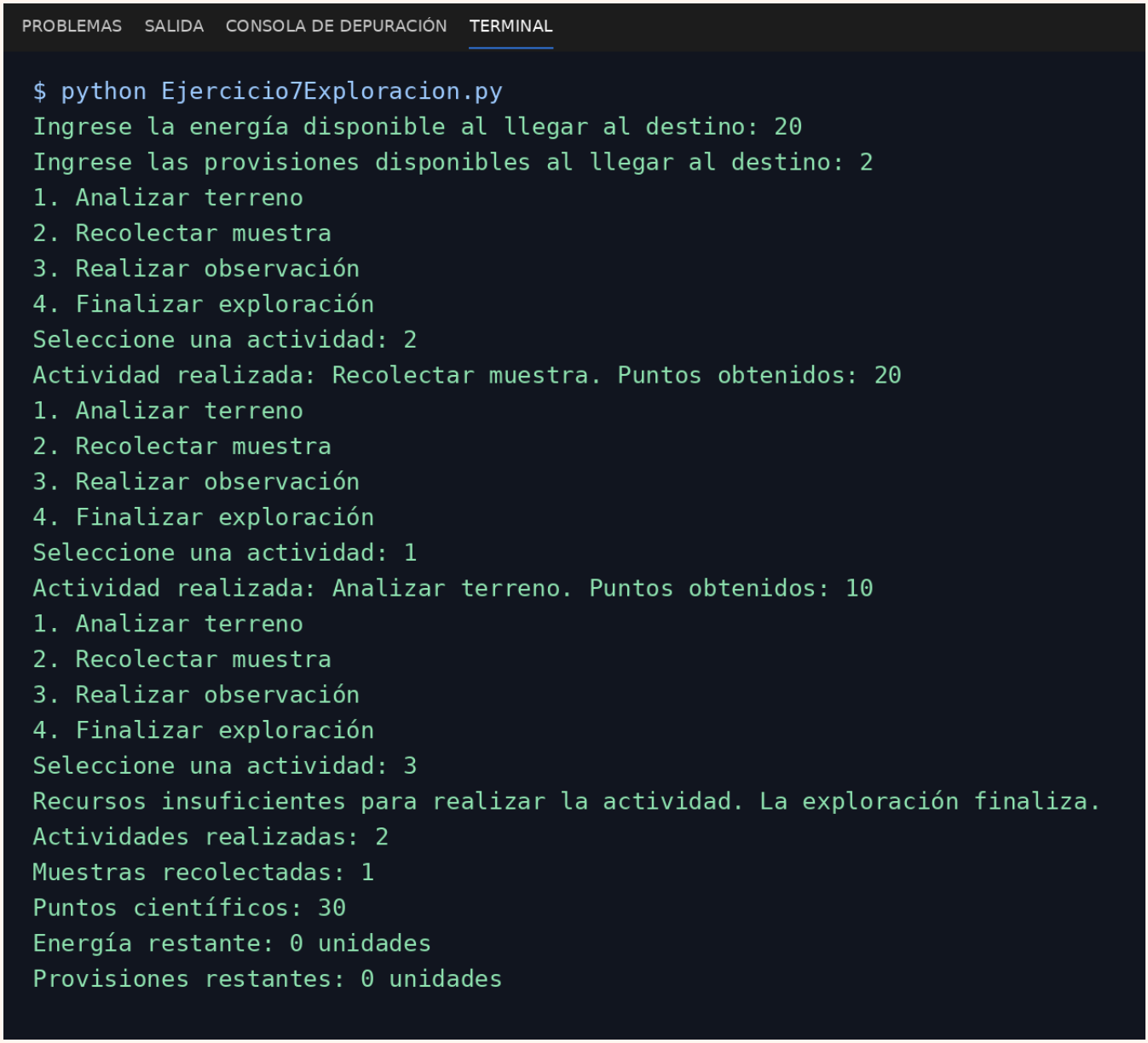
4. Casos de prueba

Primero calculé los resultados esperados y después ejecuté el programa.

ID	Ejercicio	Escenario	Datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-07-01	Ejercicio 7	Ejemplo de la consigna: dos actividades y una rechazada por energía	Energía = 20; provisiones = 2; opciones: 2, 1, 3	Recolectar muestra (20 pts); Analizar terreno (10 pts); luego Realizar observación se rechaza con “Recursos insuficientes...”. Resumen: 2 actividades, 1 muestra, 30 puntos, energía 0, provisiones 0	Igual al esperado	Exitoso
CP-07-02	Ejercicio 7	Varias actividades con acumulación, opción no válida, límite exacto y fin con opción 4	Energía = 25; provisiones = 4; opciones: 1, 7, 3, 2, 4	Analizar terreno (10 pts); 7 = “Opción no válida.”; Realizar observación (8 pts); Recolectar muestra con energía 12 exacta (20 pts); 4 = “Exploración finalizada por la tripulación.” Resumen: 3 actividades, 1 muestra, 38 puntos, energía 0, provisiones 2	Igual al esperado	Exitoso
CP-07-03	Ejercicio 7	Finalizar de inmediato: no cambia ningún valor	Energía = 100; provisiones = 5; opciones: 4	“Exploración finalizada por la tripulación.” Resumen: 0 actividades, 0 muestras, 0 puntos, energía 100, provisiones 5	Igual al esperado	Exitoso
CP-07-04	Ejercicio 7	Actividad rechazada solo por provisiones (hay energía de sobra)	Energía = 30; provisiones = 1; opciones: 2	Recolectar muestra necesita 2 provisiones y hay 1: “Recursos insuficientes...” y el menú termina. Resumen: 0 actividades, 0 muestras, 0 puntos, energía 30, provisiones 1	Igual al esperado	Exitoso
CP-07-05	Ejercicio 7	Opción 0 no válida y rechazo en el límite (energía llega a 0)	Energía = 13; provisiones = 0; opciones: 0, 1, 3, 3	0 = “Opción no válida.”; Analizar terreno (energía 5); Realizar observación con energía 5 exacta (energía 0); la segunda observación se rechaza. Resumen: 2 actividades, 0 muestras, 18 puntos, energía 0, provisiones 0	Igual al esperado	Exitoso

Capturas de la ejecución

CP-07-01

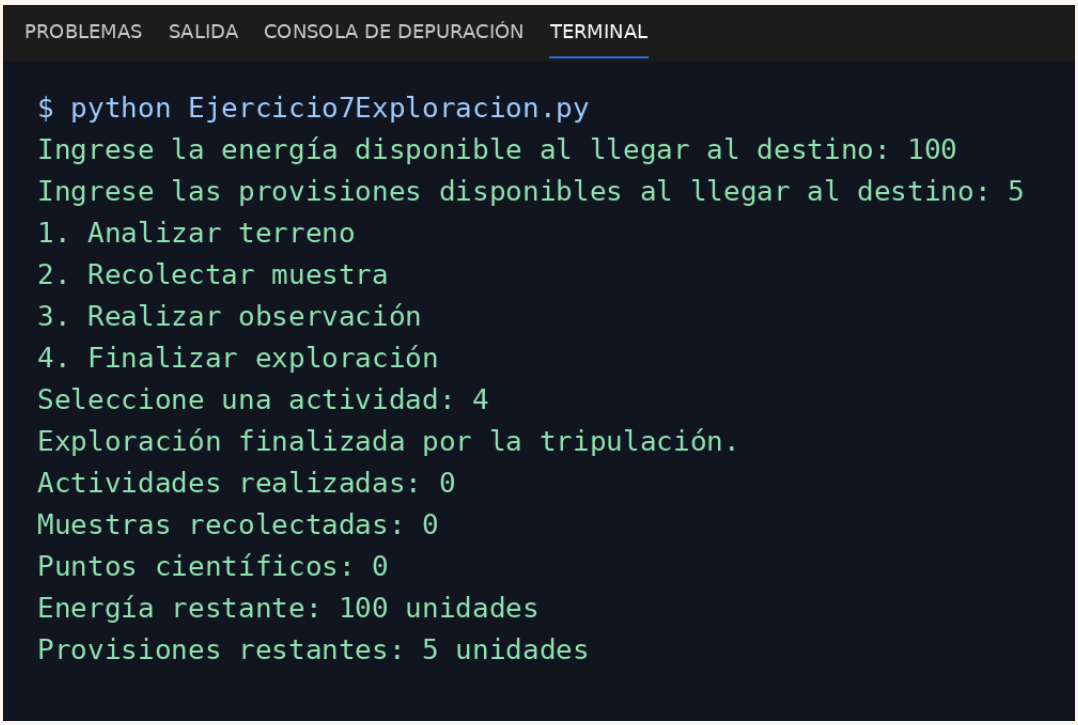


CP-07-02

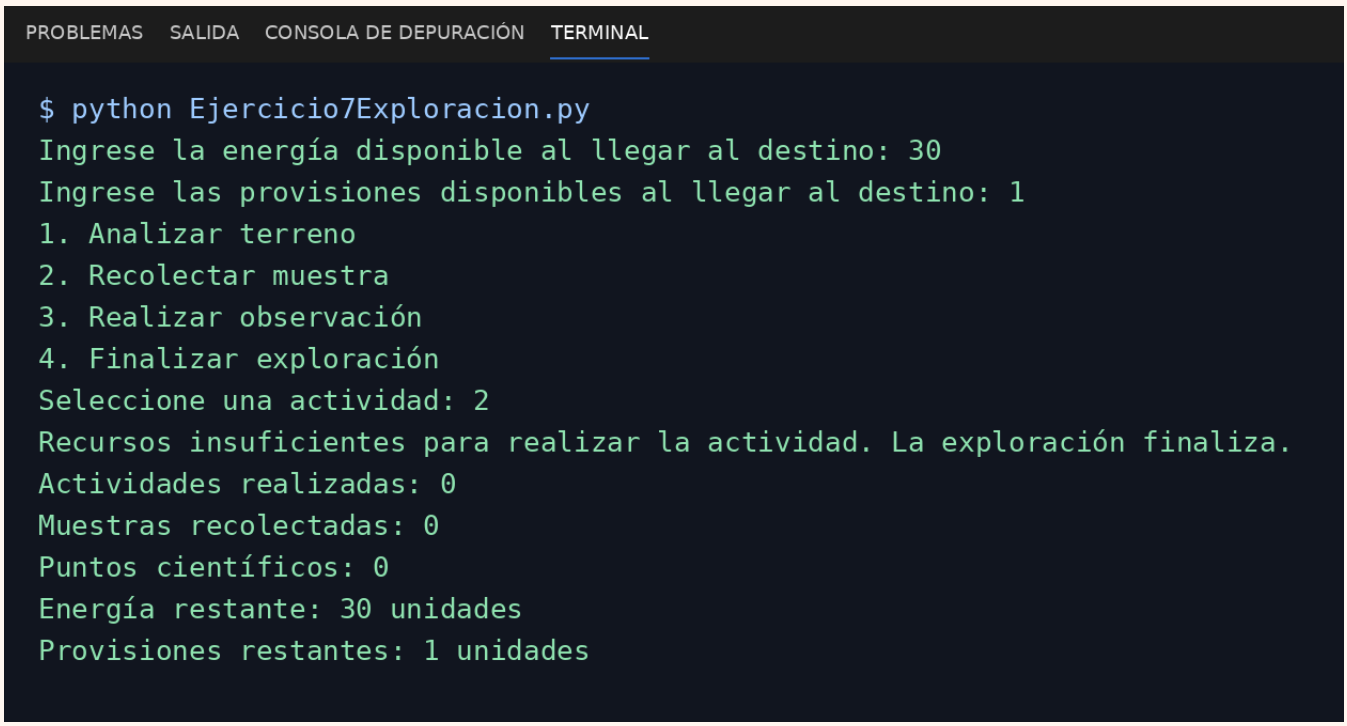
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL

```
$ python Ejercicio7Exploracion.py
Ingrese la energía disponible al llegar al destino: 25
Ingrese las provisiones disponibles al llegar al destino: 4
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una actividad: 1
Actividad realizada: Analizar terreno. Puntos obtenidos: 10
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una actividad: 7
Opción no válida.
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una actividad: 3
Actividad realizada: Realizar observación. Puntos obtenidos: 8
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una actividad: 2
Actividad realizada: Recolectar muestra. Puntos obtenidos: 20
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una actividad: 4
Exploración finalizada por la tripulación.
Actividades realizadas: 3
Muestras recolectadas: 1
Puntos científicos: 38
Energía restante: 0 unidades
Provisiones restantes: 2 unidades
```

CP-07-03



CP-07-04



CP-07-05

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL

```
$ python Ejercicio7Exploracion.py
Ingrese la energía disponible al llegar al destino: 13
Ingrese las provisiones disponibles al llegar al destino: 0
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una actividad: 0
Opción no válida.
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una actividad: 1
Actividad realizada: Analizar terreno. Puntos obtenidos: 10
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una actividad: 3
Actividad realizada: Realizar observación. Puntos obtenidos: 8
1. Analizar terreno
2. Recolectar muestra
3. Realizar observación
4. Finalizar exploración
Seleccione una actividad: 3
Recursos insuficientes para realizar la actividad. La exploración finaliza.
Actividades realizadas: 2
Muestras recolectadas: 0
Puntos científicos: 18
Energía restante: 0 unidades
Provisiones restantes: 0 unidades
```

Ejercicio 8: Evaluación del regreso y supervivencia de la tripulación

Archivo: Ejercicio8Regreso.py
Tipo de estructura: Condicional doble para elegir el caso, ciclo while (caso A) y condicionales para el menor valor (caso B).

1. Tabla de variables

Nombre	Tipo de dato	Descripción	Clasificación	Valor inicial / restricción
CONSUMO_COMBUSTIBLE_DIARIO	int	Combustible que se gasta cada día.	Constante	8
CONSUMO_OXIGENO_DIARIO	int	Oxígeno que gasta cada tripulante por día.	Constante	2
CONSUMO_ENERGIA_DIARIO	int	Energía que gasta la nave por día.	Constante	5
CONSUMO_PROVISIONES_DIARIO	int	Provisiones que come cada tripulante por día.	Constante	1
cantidad_tripulantes	int	Cantidad de tripulantes.	Entrada	Dato del usuario; entero positivo
dias_viaje	int	Días del viaje de ida (el regreso dura lo mismo).	Entrada	Dato del usuario; entero positivo
combustible	int	Combustible disponible; en el caso A se actualiza cada día.	Entrada	Dato del usuario; entero positivo
oxigeno	int	Oxígeno disponible; en el caso A se actualiza cada día.	Entrada	Dato del usuario; entero positivo
energia	int	Energía disponible; en el caso A se actualiza cada día.	Entrada	Dato del usuario; entero positivo
provisiones	int	Provisiones disponibles; en el caso A se actualizan cada día.	Entrada	Dato del usuario; entero positivo
consumo_oxigeno_dia	int	Oxígeno que gasta toda la tripulación en un día.	Intermedia	Se calcula en el programa
consumo_provisiones_dia	int	Provisiones que come toda la tripulación en un día.	Intermedia	Se calcula en el programa
combustible_regreso_necesario	int	Combustible que se necesita para volver (sin reserva).	Intermedia	Se calcula en el programa
dias_completados	int	Días del regreso que ya se completaron.	Intermedia (contador)	0
regreso_en_curso	bool	Indica si el regreso puede seguir.	Intermedia (variable de control)	True; pasa a False si falta un recurso
dias_oxigeno	int	Días completos que alcanza el oxígeno.	Intermedia	Se calcula en el caso B
dias_energia	int	Días completos que alcanza la energía.	Intermedia	Se calcula en el caso B
dias_provisiones	int	Días completos que alcanzan las provisiones.	Intermedia	Se calcula en el caso B
supervivencia_dias	int	Menor cantidad de días entre los tres recursos.	Salida	Se calcula en el caso B
recursos_limitantes	str	Nombre(s) del recurso o recursos que limitan.	Salida	Texto vacío; se arma en el caso B

2. Diagrama de flujo - Ejercicio 8

Diagrama de flujo - Ejercicio 8: Evaluación del regreso y supervivencia de la tripulación (parte 1 de 5)

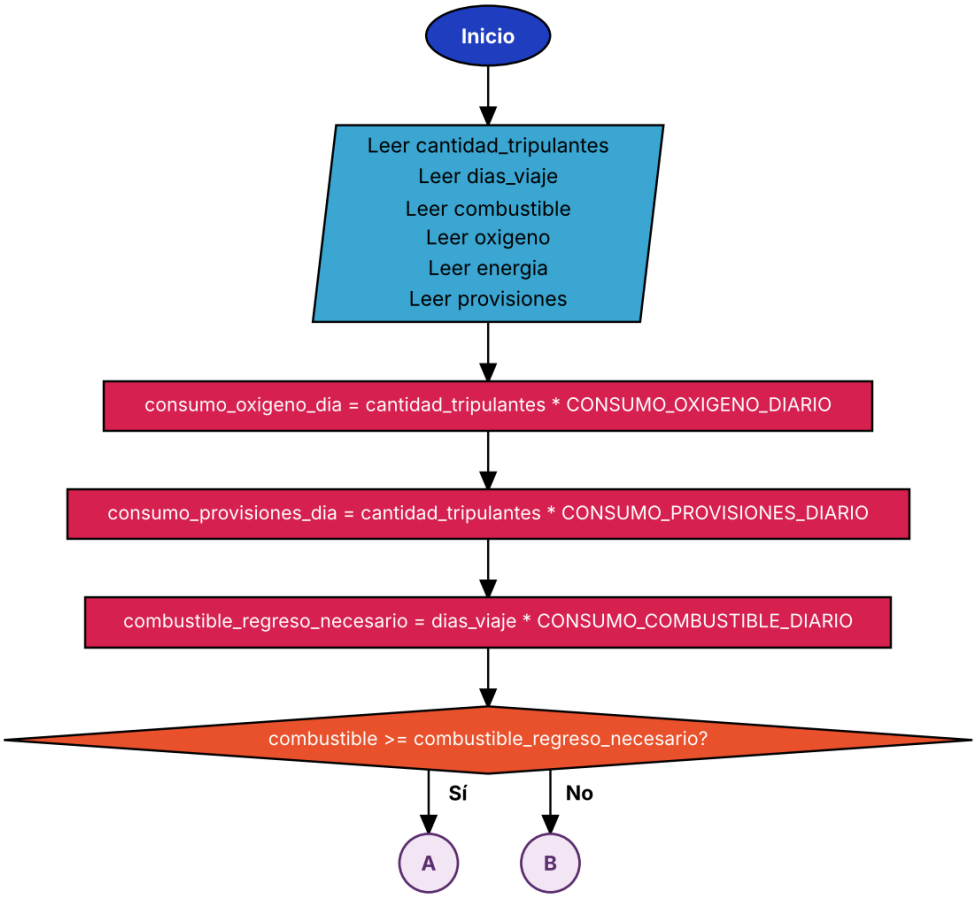


Diagrama de flujo - Ejercicio 8: Evaluación del regreso y supervivencia de la tripulación (parte 2 de 5): caso A, ciclo del regreso

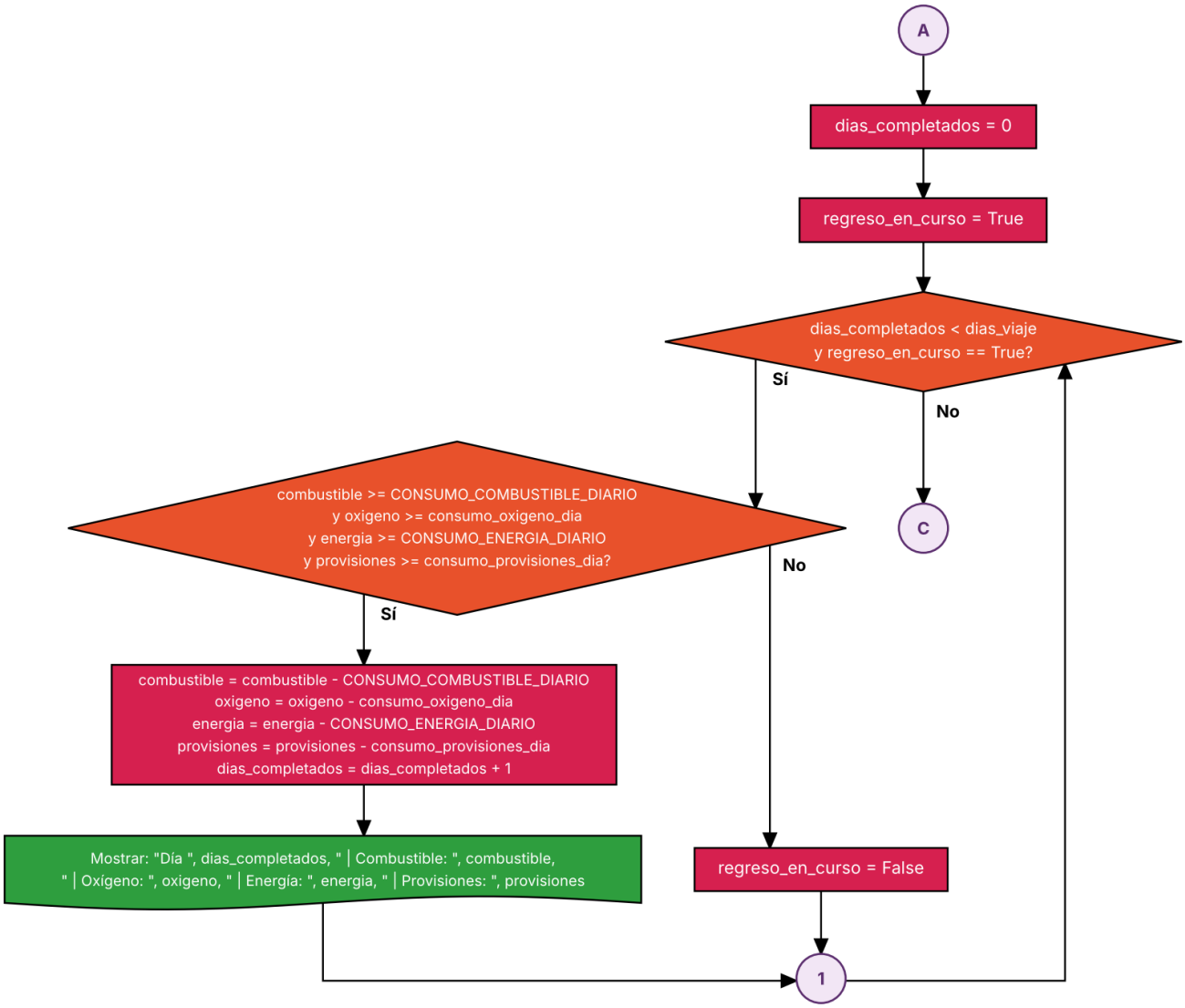


Diagrama de flujo - Ejercicio 8: Evaluación del regreso y supervivencia de la tripulación (parte 3 de 5): caso A, resultado del regreso

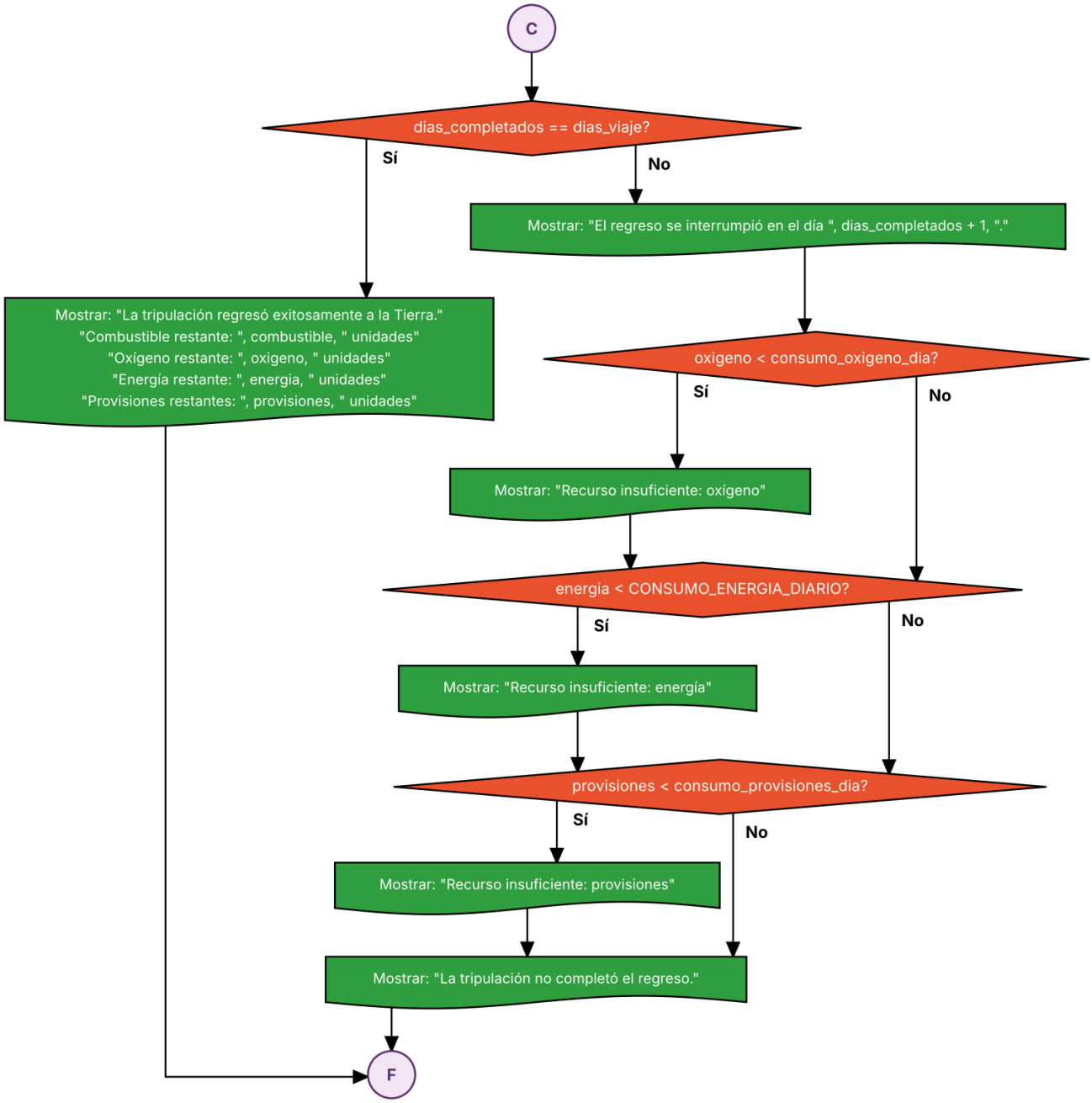


Diagrama de flujo - Ejercicio 8: Evaluación del regreso y supervivencia de la tripulación (parte 4 de 5): caso B, supervivencia

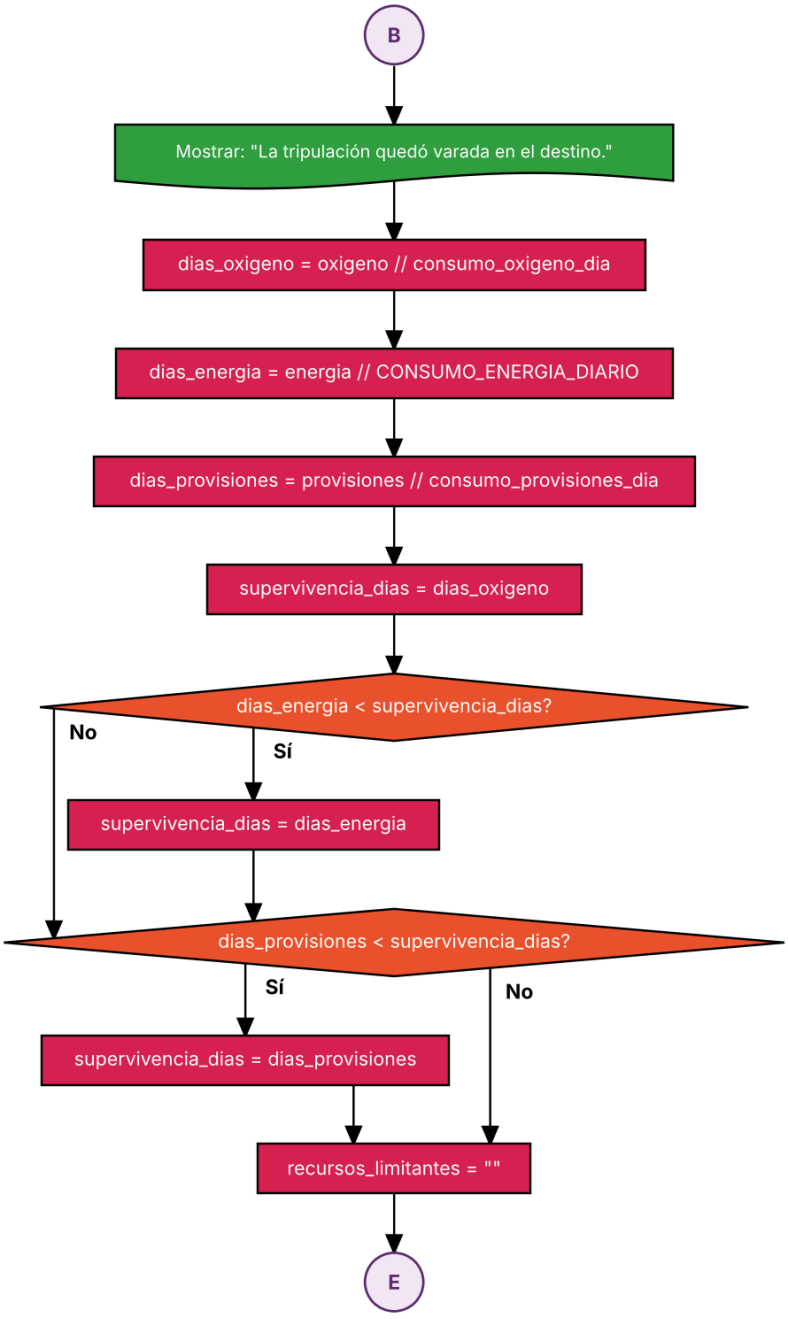
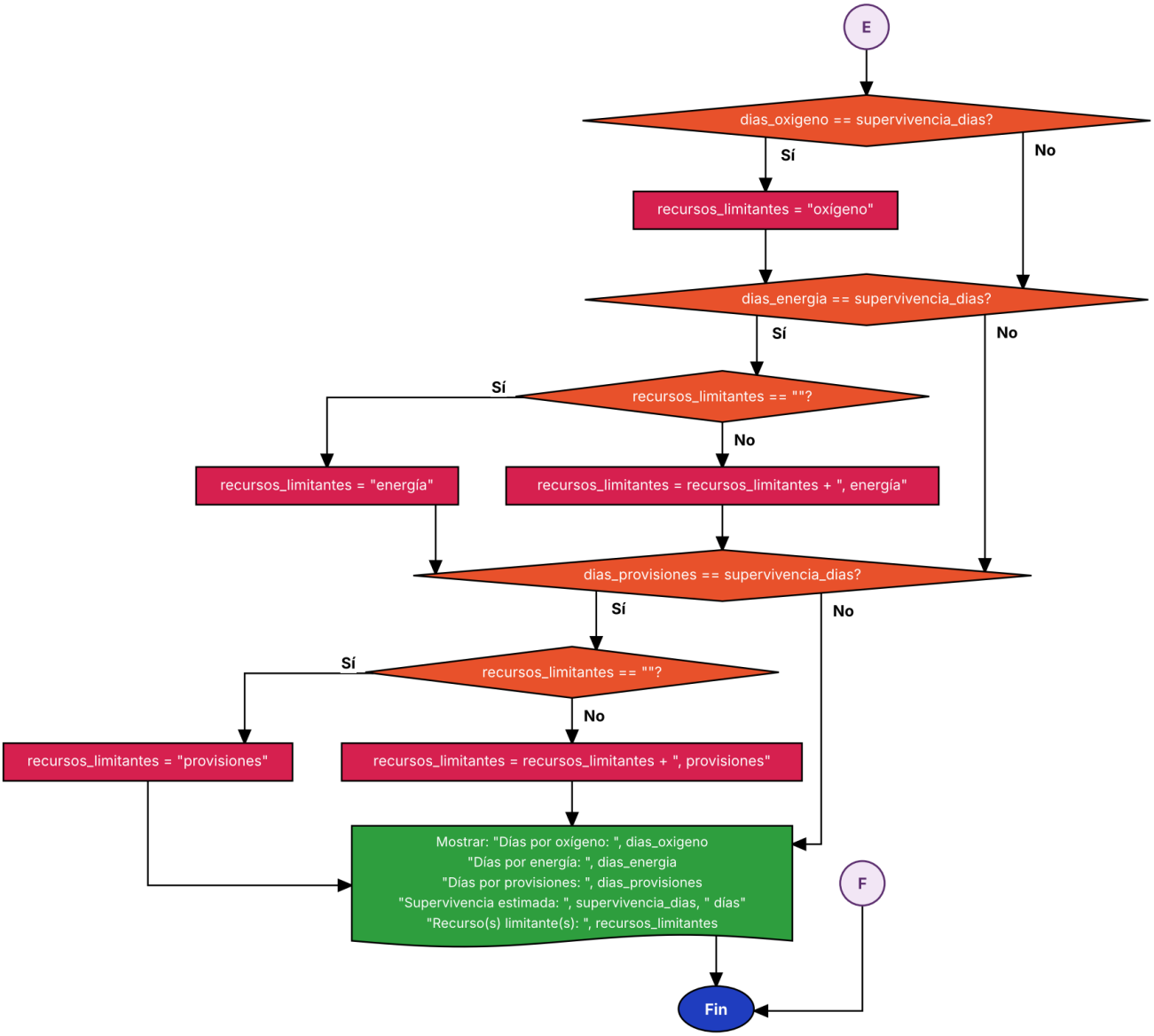


Diagrama de flujo - Ejercicio 8: Evaluación del regreso y supervivencia de la tripulación (parte 5 de 5): caso B, recursos limitantes



3. Código

Capturas del código en Visual Studio Code.

PY Ejercicio8Regreso.py

```python
# ============================================================
# Archivo: Ejercicio8Regreso.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV6
# Integrantes: Tayrin Alexa Ramírez Barrantes
# Fecha: 05/11/2026       Versión: 1.0
# Descripción: Evalúa el regreso de la nave o la supervivencia de la tripulación varada.
# ============================================================

# --- Declaración de constantes ---
# Consumo diario de cada recurso (unidades ficticias de misión).
CONSUMO_COMBUSTIBLE_DIARIO = 8
CONSUMO_OXIGENO_DIARIO = 2        # por tripulante
CONSUMO_ENERGIA_DIARIO = 5
CONSUMO_PROVISIONES_DIARIO = 1    # por tripulante

# --- Entrada de datos ---
cantidad_tripulantes = int(input("Ingrese la cantidad de tripulantes: "))
dias_viaje = int(input("Ingrese la cantidad de días del viaje de ida: "))
combustible = int(input("Ingrese el combustible disponible: "))
oxigeno = int(input("Ingrese el oxígeno disponible: "))
energia = int(input("Ingrese la energía disponible: "))
provisiones = int(input("Ingrese las provisiones disponibles: "))

# --- Procesamiento previo ---
# El consumo por tripulante se calcula una sola vez porque no cambia durante el regreso.
consumo_oxigeno_dia = cantidad_tripulantes * CONSUMO_OXIGENO_DIARIO
consumo_provisiones_dia = cantidad_tripulantes * CONSUMO_PROVISIONES_DIARIO
# En este ejercicio no se suma la reserva de emergencia.
combustible_regreso_necesario = dias_viaje * CONSUMO_COMBUSTIBLE_DIARIO

# --- Decisión principal: caso A (regresa) o caso B (queda varada) ---
if combustible >= combustible_regreso_necesario:
    # --- Caso A: simulación de los días del regreso ---
    dias_completados = 0
    regreso_en_curso = True

```

PY Ejercicio8Regreso.py

```
    # El ciclo termina cuando se completan los días o cuando un recurso no alcanza.
    while dias_completados < dias_viaje and regreso_en_curso:
        # Se verifica antes de consumir para no dejar recursos negativos.
        if (combustible >= CONSUMO_COMBUSTIBLE_DIARIO
                and oxigeno >= consumo_oxigeno_dia
                and energia >= CONSUMO_ENERGIA_DIARIO
                and provisiones >= consumo_provisiones_dia):
            combustible = combustible - CONSUMO_COMBUSTIBLE_DIARIO
            oxigeno = oxigeno - consumo_oxigeno_dia
            energia = energia - CONSUMO_ENERGIA_DIARIO
            provisiones = provisiones - consumo_provisiones_dia
            dias_completados = dias_completados + 1
            print("Día " + str(dias_completados) + " | Combustible: " + str(combustible)
                  + " | Oxígeno: " + str(oxigeno) + " | Energía: " + str(energia)
                  + " | Provisiones: " + str(provisiones))
        else:
            regreso_en_curso = False

    # --- Salida del caso A ---
    if dias_completados == dias_viaje:
        print("La tripulación regresó exitosamente a la Tierra.")
        print("Combustible restante: " + str(combustible) + " unidades")
        print("Oxígeno restante: " + str(oxigeno) + " unidades")
        print("Energía restante: " + str(energia) + " unidades")
        print("Provisiones restantes: " + str(provisiones) + " unidades")
    else:
        print("El regreso se interrumpió en el día " + str(dias_completados + 1) + ".")
        # El combustible no se revisa aquí porque ya se verificó que alcanzaba para todo el regreso.
        if oxigeno < consumo_oxigeno_dia:
            print("Recurso insuficiente: oxígeno")
        if energia < CONSUMO_ENERGIA_DIARIO:
            print("Recurso insuficiente: energía")
        if provisiones < consumo_provisiones_dia:
            print("Recurso insuficiente: provisiones")
        print("La tripulación no completó el regreso.")
else:
```

PY Ejercicio8Regreso.py

```python
    # --- Caso B: tripulación varada, se estiman los días de supervivencia ---
    print("La tripulación quedó varada en el destino.")
    dias_oxigeno = oxigeno // consumo_oxigeno_dia
    dias_energia = energia // CONSUMO_ENERGIA_DIARIO
    dias_provisiones = provisiones // consumo_provisiones_dia

    # El menor de los tres valores se obtiene con condicionales (sin min()).
    supervivencia_dias = dias_oxigeno
    if dias_energia < supervivencia_dias:
        supervivencia_dias = dias_energia
    if dias_provisiones < supervivencia_dias:
        supervivencia_dias = dias_provisiones

    # Si hay empate, se agregan todos los recursos con el valor mínimo, separados por coma.
    recursos_limitantes = ""
    if dias_oxigeno == supervivencia_dias:
        recursos_limitantes = "oxígeno"
    if dias_energia == supervivencia_dias:
        if recursos_limitantes == "":
            recursos_limitantes = "energía"
        else:
            recursos_limitantes = recursos_limitantes + ", energía"
    if dias_provisiones == supervivencia_dias:
        if recursos_limitantes == "":
            recursos_limitantes = "provisiones"
        else:
            recursos_limitantes = recursos_limitantes + ", provisiones"

    print("Días por oxígeno: " + str(dias_oxigeno))
    print("Días por energía: " + str(dias_energia))
    print("Días por provisiones: " + str(dias_provisiones))
    print("Supervivencia estimada: " + str(supervivencia_dias) + " días")
    print("Recurso(s) limitante(s): " + recursos_limitantes)
```

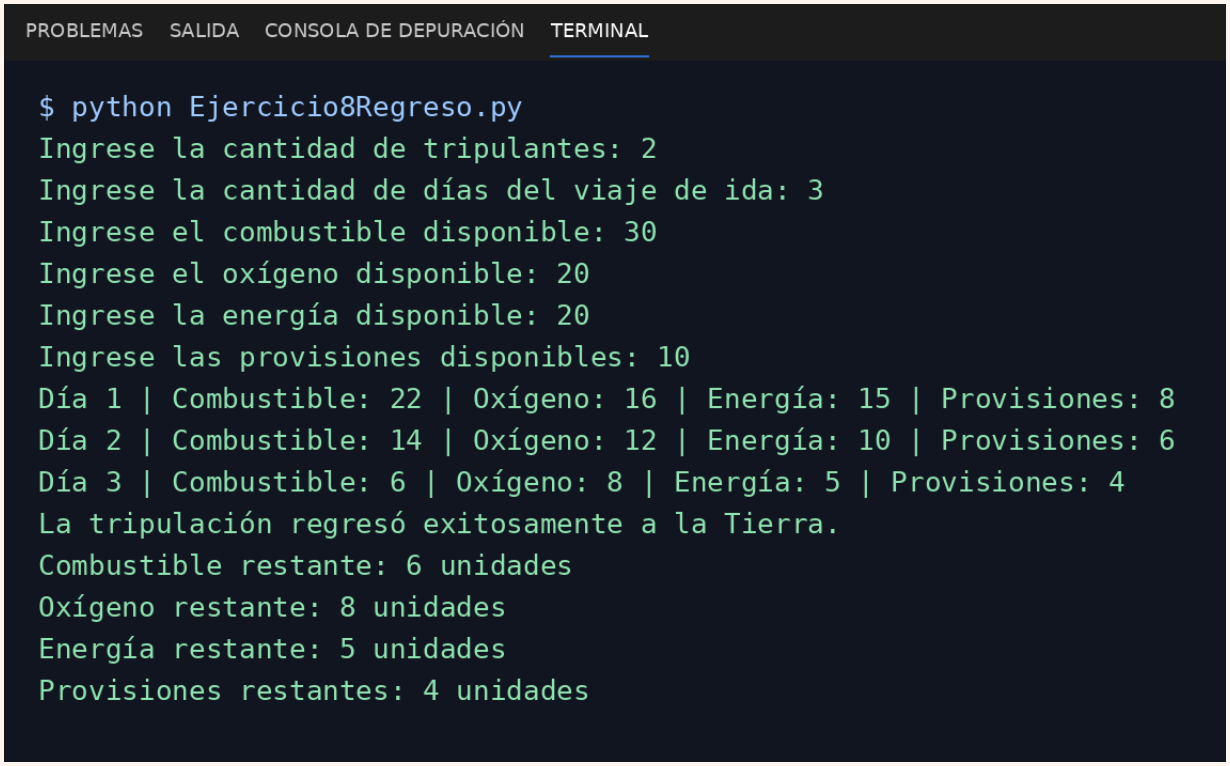
4. Casos de prueba

Primero calculé los resultados esperados y después ejecuté el programa.

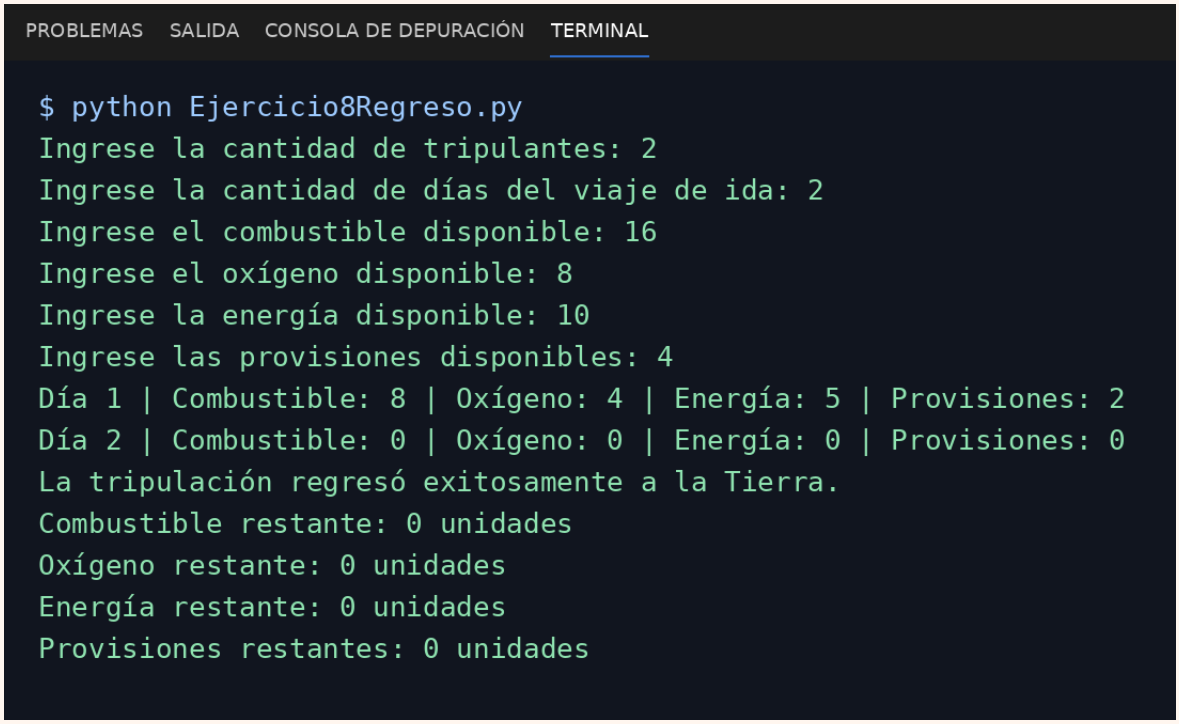
ID	Ejercicio	Escenario	Datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-08-01	Ejercicio 8	Regreso completo (ejemplo de la consigna)	Tripulantes = 2; días = 3; combustible = 30; oxígeno = 20; energía = 20; provisiones = 10	Días 1 a 3 con su estado (día 3: 6, 8, 5, 4); “La tripulación regresó exitosamente a la Tierra.”; restantes: combustible 6, oxígeno 8, energía 5, provisiones 4	Igual al esperado	Exitoso
CP-08-02	Ejercicio 8	Regreso exactamente justo: combustible igual al necesario y todo queda en cero	Tripulantes = 2; días = 2; combustible = 16; oxígeno = 8; energía = 10; provisiones = 4	Combustible 16 = 2×8 → caso A. Día 1: 8, 4, 5, 2; día 2: 0, 0, 0, 0. “La tripulación regresó exitosamente a la Tierra.” con todos los restantes en 0	Igual al esperado	Exitoso
CP-08-03	Ejercicio 8	Regreso interrumpido por un solo recurso (oxígeno)	Tripulantes = 2; días = 3; combustible = 30; oxígeno = 10; energía = 20; provisiones = 10	Día 1: 22, 6, 15, 8; día 2: 14, 2, 10, 6; día 3 no se puede: “El regreso se interrumpió en el día 3.”; “Recurso insuficiente: oxígeno”; “La tripulación no completó el regreso.”	Igual al esperado	Exitoso
CP-08-04	Ejercicio 8	Regreso interrumpido por más de un recurso (energía y provisiones)	Tripulantes = 2; días = 3; combustible = 40; oxígeno = 20; energía = 12; provisiones = 4	Día 1: 32, 16, 7, 2; día 2: 24, 12, 2, 0; día 3 no se puede: “El regreso se interrumpió en el día 3.”; “Recurso insuficiente: energía”; “Recurso insuficiente: provisiones”; “La tripulación no completó el regreso.”	Igual al esperado	Exitoso
CP-08-05	Ejercicio 8	Tripulación varada con un solo recurso limitante (provisiones)	Tripulantes = 3; días = 5; combustible = 10; oxígeno = 60; energía = 100; provisiones = 24	Combustible 10 < 40 → caso B. Días por oxígeno 60//6 = 10; por energía 100//5 = 20; por provisiones 24//3 = 8. Supervivencia: 8 días; limitante: provisiones	Igual al esperado	Exitoso
CP-08-06	Ejercicio 8	Tripulación varada con empate entre oxígeno y energía (ejemplo de la consigna)	Tripulantes = 2; días = 5; combustible = 10; oxígeno = 20; energía = 25; provisiones = 12	Combustible 10 < 40 → caso B. Días por oxígeno 20//4 = 5; por energía 25//5 = 5; por provisiones 12//2 = 6. Supervivencia: 5 días; limitantes: oxígeno, energía	Igual al esperado	Exitoso
CP-08-07	Ejercicio 8	Límite del combustible: falta 1 unidad y hay empate entre los tres recursos	Tripulantes = 2; días = 2; combustible = 15; oxígeno = 8; energía = 10; provisiones = 4	Combustible 15 < 16 → caso B. Días por oxígeno 8//4 = 2; por energía 10//5 = 2; por provisiones 4//2 = 2. Supervivencia: 2 días; limitantes: oxígeno, energía, provisiones	Igual al esperado	Exitoso

Capturas de la ejecución

CP-08-01



CP-08-02



CP-08-03

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL

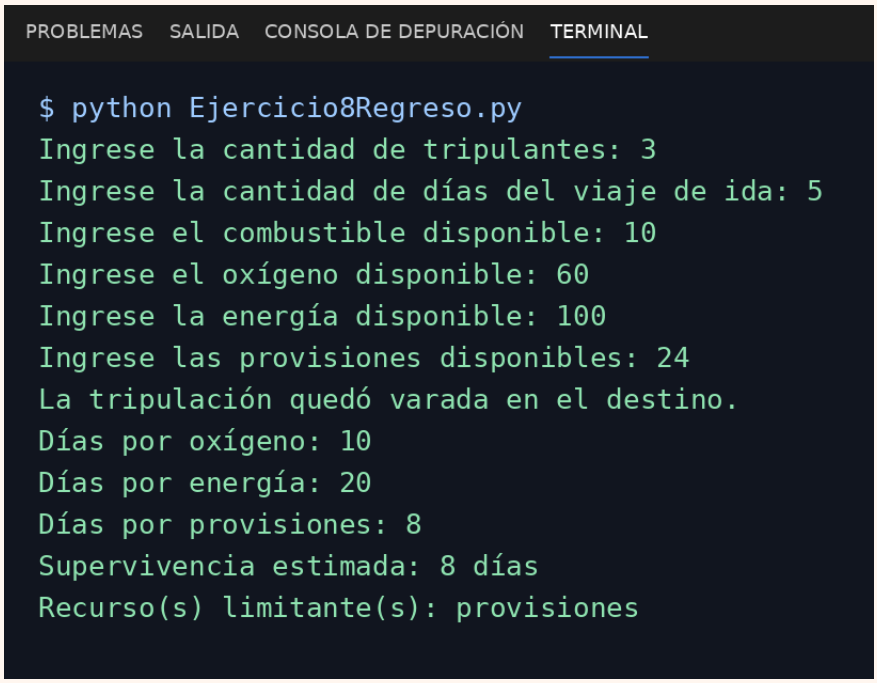
```
$ python Ejercicio8Regreso.py
Ingrese la cantidad de tripulantes: 2
Ingrese la cantidad de días del viaje de ida: 3
Ingrese el combustible disponible: 30
Ingrese el oxígeno disponible: 10
Ingrese la energía disponible: 20
Ingrese las provisiones disponibles: 10
Día 1 | Combustible: 22 | Oxígeno: 6 | Energía: 15 | Provisiones: 8
Día 2 | Combustible: 14 | Oxígeno: 2 | Energía: 10 | Provisiones: 6
El regreso se interrumpió en el día 3.
Recurso insuficiente: oxígeno
La tripulación no completó el regreso.
```

CP-08-04

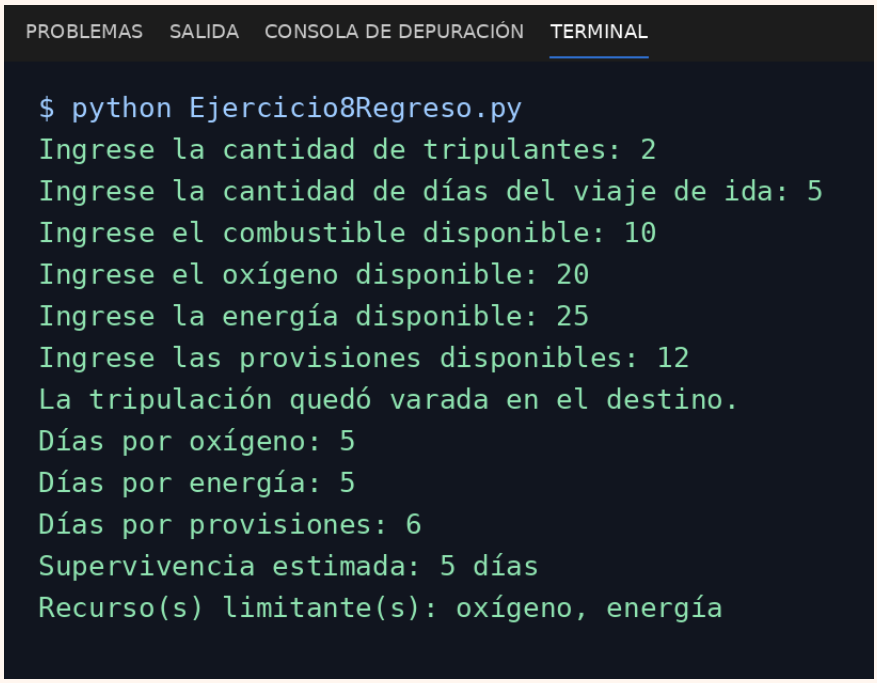
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL

```
$ python Ejercicio8Regreso.py
Ingrese la cantidad de tripulantes: 2
Ingrese la cantidad de días del viaje de ida: 3
Ingrese el combustible disponible: 40
Ingrese el oxígeno disponible: 20
Ingrese la energía disponible: 12
Ingrese las provisiones disponibles: 4
Día 1 | Combustible: 32 | Oxígeno: 16 | Energía: 7 | Provisiones: 2
Día 2 | Combustible: 24 | Oxígeno: 12 | Energía: 2 | Provisiones: 0
El regreso se interrumpió en el día 3.
Recurso insuficiente: energía
Recurso insuficiente: provisiones
La tripulación no completó el regreso.
```

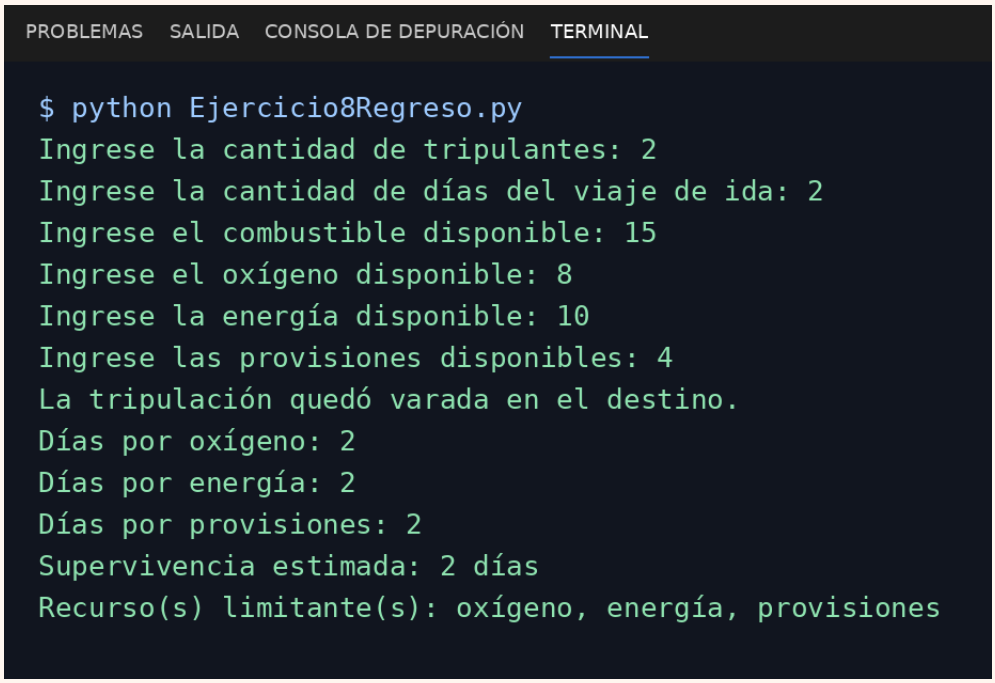
CP-08-05



CP-08-06



CP-08-07


PROBLEMAS
SALIDA
CONSOLA DE DEPURACIÓN
TERMINAL
$ python Ejercicio8Regreso.py
Ingrese la cantidad de tripulantes: 2
Ingrese la cantidad de días del viaje de ida: 2
Ingrese el combustible disponible: 15
Ingrese el oxígeno disponible: 8
Ingrese la energía disponible: 10
Ingrese las provisiones disponibles: 4
La tripulación quedó varada en el destino.
Días por oxígeno: 2
Días por energía: 2
Días por provisiones: 2
Supervivencia estimada: 2 días
Recurso(s) limitante(s): oxígeno, energía, provisiones

Enlace al repositorio Git

Repositorio: https://github.com/Tay20RB-ux/mision-orbita

Estructura del repositorio:

```
mision-orbita/
├── README.md
├── avance1/
│   ├── Ejercicio1Recursos.py
│   ├── Ejercicio2Combustible.py
│   ├── Ejercicio3SoporteTripulacion.py
│   └── Ejercicio4Autorizacion.py
├── avance2/
│   ├── Ejercicio7Exploracion.py
│   └── Ejercicio8Regreso.py
└── documentacion/
    ├── Avance1.pdf
    ├── Ejercicio7Exploracion.pdf
    └── Ejercicio8Regreso.pdf
```